Cours Magistral Complet

Apprentissage par Renforcement (Reinforcement Learning)

M1 Intelligence Artificielle

2026

Table des matières

1 Introduction au Reinforcement Learning **2**
1.1 Le Concept Fondamental 2
1.2 Comparaison avec les Autres Paradigmes 2
1.2.1 Vs Apprentissage Supervisé (Supervised Learning) 2
1.2.2 Vs Apprentissage Non-Supervisé (Unsupervised Learning) 3
1.3 Le Cycle d'Interaction Agent-Environnement 3

2 Formalisation Mathématique : Les MDP **4**
2.1 Définition d'un MDP 4
2.2 La Propriété de Markov 5
2.3 L'Objectif : Le Retour (Return) 5

3 Politiques et Fonctions de Valeur **6**
3.1 La Politique (Policy π) 6
3.2 Les Fonctions de Valeur (Value Functions) 6
3.2.1 1. La State-Value Function $V^\pi(s)$ 6
3.2.2 2. La Action-Value Function $Q^\pi(s, a)$ 6
3.3 Les Équations de Bellman 7
3.3.1 Équation de Bellman pour V^π (Expectation Equation) 7
3.3.2 Équation de Bellman pour V^* (Optimality Equation) 7

4 Résolution "Model-Based" (Programmation Dynamique) **8**
4.1 Exemple Illustratif : Le Monde A-B-C 8
4.2 Algorithme 1 : Value Iteration 8
4.3 Algorithme 2 : Policy Iteration 9
4.4 Note Technique pour le TP : Accéder à la Dynamique P 10

5 Résolution "Model-Free" (Apprentissage Réel) **11**
5.1 Monte Carlo vs Temporal Difference (TD) 11
5.2 Algorithmes de Contrôle (Q-Learning et SARSA) 11
5.2.1 SARSA (On-Policy) 11
5.2.2 Q-Learning (Off-Policy) 12

6 Le Dilemme Exploration-Exploitation **13**
6.1 La Solution : Epsilon-Greedy 13

7 Mise en Pratique : Code Commenté **14**

Chapitre 1

Introduction au Reinforcement Learning

1.1 Le Concept Fondamental

L'Apprentissage par Renforcement (ou *Reinforcement Learning* - RL) est l'un des trois paradigmes majeurs du Machine Learning. Contrairement aux approches classiques où l'on fournit des données corrigées à la machine, le RL s'intéresse à la **prise de décision séquentielle**.

Le principe est inspiré de la psychologie comportementale : une entité (l'**Agent**) apprend à agir dans un monde (l'**Environnement**) en observant les conséquences de ses actions.

— Si l'issue est positive, l'action est renforcée (récompensée).

— Si l'issue est négative, l'action est découragée (punie).

Intuition & Compréhension : L'Analogie de l'Enfant

Imaginez un enfant apprenant à marcher. Personne ne lui donne une équation différentielle de la gravité ou ne lui dit "contracte le muscle quadriceps de 12%". L'enfant essaie de se lever, tombe (douleur/échec), essaie de modifier son appui, réussit à tenir debout (joie/succès). C'est exactement ce que fait un agent RL : il apprend par **essais et erreurs** (Trial and Error).

1.2 Comparaison avec les Autres Paradigmes

Il est crucial de situer le RL par rapport aux autres méthodes d'IA pour comprendre sa spécificité.

1.2.1 Vs Apprentissage Supervisé (Supervised Learning)

En apprentissage supervisé (ex : classification d'images), un "professeur" fournit la réponse correcte pour chaque exemple (Image de chat → Label "Chat"). L'objectif est de minimiser l'erreur entre la prédiction et la vérité terrain.

— **Supervisé** : "Voici la bonne réponse, apprends-la."

— **RL** : "Voici une note sur 20 après que tu aies agi. Débrouille-toi pour comprendre ce qui était bien ou mal."

Le RL est plus difficile car le signal de feedback (la récompense) est souvent **rare** et **retardé**. Si vous jouez aux échecs, vous ne savez si votre premier coup était bon qu'à la toute fin de la partie.

1.2.2 Vs Apprentissage Non-Supervisé (Unsupervised Learning)

L'apprentissage non-supervisé cherche des structures cachées dans des données (clustering). Le RL, lui, a un but explicite : **maximiser une récompense**. Il ne cherche pas juste à comprendre les données, il cherche à **contrôler** un processus.

1.3 Le Cycle d'Interaction Agent-Environnement

Tout problème de RL se formalise par une boucle de rétroaction infinie entre deux entités.

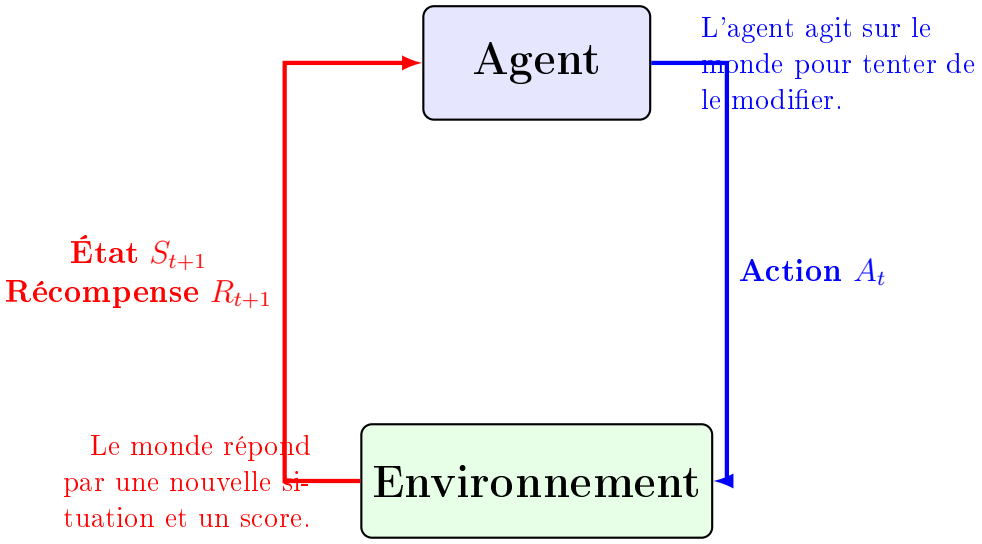


FIGURE 1.1 – La boucle Agent-Environnement. À chaque pas de temps t, l'agent reçoit S_t, joue A_t, et reçoit R_{t+1}, S_{t+1}.

L'hypothèse fondamentale est que l'agent n'a pas le contrôle total sur l'environnement (il y a une part d'incertitude), mais il peut l'influencer.

Chapitre 2

Formalisation Mathématique : Les MDP

Pour résoudre informatiquement ces problèmes, nous devons définir un cadre mathématique rigoureux. Ce cadre est le **Processus de Décision de Markov** (Markov Decision Process - MDP).

2.1 Définition d'un MDP

Un MDP est défini par un tuple de 5 éléments : (S, A, P, R, γ). Détaillons chacun d'eux avec précision.

Formalisme Mathématique : Composants du MDP

1. S **(State Space)** : L'ensemble de tous les états possibles. C'est la description complète de la situation à un instant t. Ex : La position de toutes les pièces sur un échiquier.
2. A **(Action Space)** : L'ensemble de toutes les actions que l'agent peut effectuer. Ex : "Bouger Pion A en C3".
3. P **(Probabilité de Transition)** : La dynamique du monde. C'est la probabilité de passer de l'état s à l'état s' en ayant fait l'action a.

   $$P(s'|s, a) = \mathbb{P}(S_{t+1} = s'|S_t = s, A_t = a)$$

   Ceci modélise l'incertitude (le vent qui dévie le robot, l'adversaire aux échecs...).
4. R **(Fonction de Récompense)** : Le feedback immédiat reçu après une action.

   $$R(s, a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$$
5. γ **(Facteur d'Escompte / Discount Factor)** : Un nombre entre 0 et 1 qui détermine l'importance du futur.

2.2 La Propriété de Markov

Pourquoi appelle-t-on cela "Markovien" ? C'est une hypothèse simplificatrice très forte mais nécessaire. Un état est dit **Markov** si "le futur est indépendant du passé sachant le présent".

Autrement dit, pour prendre la bonne décision, l'agent a juste besoin de connaître l'état actuel S_t. Il n'a pas besoin de se souvenir de tout l'historique $S_0, S_1, \ldots$. L'état actuel résume toute l'information utile.

$$\mathbb{P}(S_{t+1}|S_t) = \mathbb{P}(S_{t+1}|S_1, S_2, \ldots, S_t)$$

2.3 L'Objectif : Le Retour (Return)

Le but de l'agent n'est pas de maximiser la récompense immédiate (sinon il ne ferait jamais d'effort dont la récompense est lointaine, comme étudier pour un examen). Son but est de maximiser la somme cumulée des récompenses sur le long terme.

On appelle cette somme le **Retour** (G_t).

Formalisme Mathématique : Le Retour Escompté (Discounted Return)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Le facteur γ est crucial :

- Si $\gamma \to 0$: L'agent est "myope". Il ne s'intéresse qu'à R_{t+1}.
- Si $\gamma \to 1$: L'agent est "prévoyant". Il accorde autant d'importance au futur lointain qu'au présent.

Mathématiquement, $\gamma < 1$ assure aussi que la somme infinie converge vers une valeur finie.

Chapitre 3

Politiques et Fonctions de Valeur

C'est le cœur de la théorie du RL. Comment quantifier si une situation est "bonne" ou "mauvaise" ?

3.1 La Politique (Policy π)

La politique est le "cerveau" de l'agent. C'est la fonction qui décide quelle action prendre dans un état donné.

— **Politique Déterministe** : $a = \pi(s)$. Pour un état donné, l'action est toujours la même.

— **Politique Stochastique** : $\pi(a|s) = \mathbb{P}(A_t = a|S_t = s)$. L'agent choisit une action selon une probabilité. (Ex : 80% Avancer, 20% Reculer). C'est crucial pour l'exploration.

L'objectif du RL est de trouver la **Politique Optimale** π^* qui maximise le retour espéré.

3.2 Les Fonctions de Valeur (Value Functions)

Pour trouver la meilleure politique, l'agent doit être capable d'estimer la "valeur" de chaque état. Il existe deux fonctions fondamentales pour cela.

3.2.1 1. La State-Value Function $V^\pi(s)$

La fonction $V^\pi(s)$ répond à la question : **"Si je suis dans l'état s et que je continue avec ma stratégie habituelle π, combien de points je vais gagner en moyenne jusqu'à la fin ?"**

$$V^\pi(s) = \mathbb{E}_\pi[G_t|S_t = s]$$

3.2.2 2. La Action-Value Function $Q^\pi(s, a)$

La fonction $Q^\pi(s, a)$ est plus précise. Elle répond à : **"Si je suis dans l'état s, que je force l'action a maintenant (même si ma politique disait autre chose), et qu'ensuite je reprends ma stratégie habituelle π, combien je gagne ?"**

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Cette fonction Q est fondamentale pour les algorithmes *Model-Free* (comme le Q-Learning) car elle permet de comparer les actions directement sans connaître la dynamique de l'environnement.

3.3 Les Équations de Bellman

Richard Bellman (1957) a formulé une récursivité qui est la clé de tout le RL. Il est fondamental de distinguer deux types d'équations.

3.3.1 Équation de Bellman pour V^π (Expectation Equation)

Cette équation décrit la valeur d'un état **sous une politique donnée** π. Elle est linéaire (une simple moyenne pondérée).

Formalisme Mathématique : Bellman Expectation Equation

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right]$$

Elle signifie : "Ma valeur actuelle est la moyenne de ce que je vais gagner tout de suite + la valeur de là où j'atterris".

3.3.2 Équation de Bellman pour V^* (Optimality Equation)

Cette équation décrit la valeur de l'état **si on joue parfaitement**. Elle contient un opérateur **max**, ce qui la rend non-linéaire.

Formalisme Mathématique : Bellman Optimality Equation

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s,a) \left[R(s,a,s') + \gamma V^*(s') \right]$$

Elle signifie : "La valeur optimale d'un état est le gain que je peux espérer si je choisis la *meilleure* action possible à partir de maintenant." C'est cette équation que l'on cherche à résoudre pour "gagner" le jeu.

Le lien entre V^* et Q^* est immédiat :

$$V^*(s) = \max_a Q^*(s,a)$$

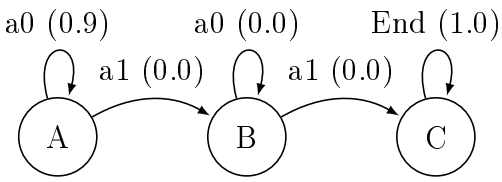
Chapitre 4

Résolution "Model-Based" (Programmation Dynamique)

Si l'agent connaît parfaitement le modèle du monde (il connaît les fonctions de transition P et de récompense R), il n'a pas besoin d'explorer physiquement. Il peut planifier. C'est le domaine de la **Programmation Dynamique (DP)**. Le principe est de décomposer le problème complexe en sous-problèmes récursifs (les états).

4.1 Exemple Illustratif : Le Monde A-B-C

Considérons un mini-monde à 3 états pour illustrer les calculs.



Légende : État A, B, C. Flèches = Actions. (x) = Probabilité ou Récompense.

4.2 Algorithme 1 : Value Iteration

Le but est de trouver $V^*(s)$ directement en appliquant l'équation d'optimalité de Bellman de manière itérative. On transforme l'égalité en une règle de mise à jour.

Formalisme Mathématique : Règle de Mise à jour (Value Iteration)

$$V_{k+1}(s) \leftarrow \max_{a \in A} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_k(s')]$$

Exemple de Calcul Pas-à-Pas

Supposons $\gamma = 0.9$ et initialisons $V_0(s) = 0$ pour tout s. **Objectif** : Atteindre C (Reward=1) depuis A.

— **Itération 1** : Pour l'état C (But), disons qu'il donne une récompense de 1.

$$V_1(C) = 1 + 0.9 \times 0 = 1$$

Pour A et B, sans récompense immédiate, ça reste 0.

— **Itération 2** : L'état B peut aller vers C.

$$V_2(B) = \max(P(C|B, a1)[R + \gamma V_1(C)]) = 1.0 \times [0 + 0.9 \times 1] = 0.9$$

L'état A ne voit pas encore C. $V_2(A) = 0$.

— **Itération 3** : L'état A peut aller vers B.

$$V_3(A) = \max(P(B|A, a1)[R + \gamma V_2(B)]) = 1.0 \times [0 + 0.9 \times 0.9] = 0.81$$

On voit que l'information de la récompense se propage ("backpropagates") depuis le but vers les états précédents à chaque itération. Après convergence, $V(A) = 0.81, V(B) = 0.9, V(C) = 1$. La politique optimale est évidente : suivre le gradient de valeur (A vers B vers C).

4.3 Algorithme 2 : Policy Iteration

Parfois, chercher la valeur précise (ϵ-convergence) est trop long alors que la politique, elle, ne change plus. L'algorithme *Policy Iteration* sépare le problème en deux étapes distinctes.

1. **Policy Evaluation** : On fixe une politique π. On calcule $V^\pi(s)$ précisement (résolution de système linéaire ou itérations simplifiées).

$$V_{k+1}(s) = \sum_{s'} P(s'|s, \pi(s))[R + \gamma V_k(s')]$$

2. **Policy Improvement** : On rend la politique "gloutonne" (greedy) par rapport aux valeurs calculées.

$$\pi_{new}(s) = \arg\max_a \sum_{s'} P(s'|s, a)[R + \gamma V^\pi(s')]$$

On répète ces deux étapes jusqu'à ce que la politique ne change plus ($\pi_{new} = \pi_{old}$).

Intuition & Compréhension : Différence VI vs PI

— **Value Iteration (VI)** : Une seule étape de "max" à chaque mise à jour. C'est plus "brutal", on met tout à jour en même temps.

— **Policy Iteration (PI)** : On prend le temps d'évaluer parfaitement une stratégie avant de la changer. Converge souvent en moins d'itérations, mais chaque itération est plus coûteuse.

4.4 Note Technique pour le TP : Accéder à la Dynamique P

Pour implémenter ces algorithmes sur un ordinateur (ex : avec la librairie `gymnasium` et l'environnement `FrozenLake`), vous avez besoin d'accéder concrètement aux probabilités de transition $P(s'|s,a)$.

Dans `FrozenLake`, cet accès est donné par l'attribut `env.P`. C'est un dictionnaire de dictionnaires : `env.P[state][action]` qui retourne une liste de tuples.

```
# Structure de env.P[state][action] :
# C'est une liste de tuples (probabilite, etat_suivant, recompense, termine)

transitions = env.P[state][action]

for prob, next_state, reward, terminated in transitions:
    # prob       -> P(s'|s,a)
    # next_state -> s'
    # reward     -> R(s,a,s') (Attention : parfois r est donne, parfois il depend de l'arrivee)
```

C'est sur cette liste que vous devrez itérer pour calculer vos sommes mathématiques :

$$\sum_{s'} P(s'|s,a)[\ldots] \quad \longrightarrow \quad \texttt{for prob, ... in env.P[s][a]: ...}$$

Chapitre 5

Résolution "Model-Free" (Apprentissage Réel)

Dans la vraie vie, on ne connaît pas les probabilités de transition P. On ne peut pas calculer d'espérances. On doit **apprendre en agissant**. C'est le domaine du **Model-Free RL**.

5.1 Monte Carlo vs Temporal Difference (TD)

Il y a deux façons d'apprendre sans modèle :

Monte Carlo (MC) L'agent joue un épisode complet jusqu'à la fin (Game Over). Il regarde le score total obtenu (G_t). Il met à jour les états visités en disant "Tiens, cet état m'a mené à une victoire, je vais augmenter sa valeur".

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

Problème : Il faut attendre la fin de l'épisode (impossible pour les tâches continues) et la variance est élevée.

Temporal Difference (TD) L'agent n'attend pas la fin. Il fait un pas, reçoit une récompense R_{t+1}, se retrouve en S_{t+1}, et utilise sa propre estimation actuelle de $V(S_{t+1})$ pour mettre à jour $V(S_t)$. C'est le principe du **Bootstrapping** (apprendre d'une estimation).

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{Cible TD}} - V(S_t)]$$

Avantage : Apprentissage pas-à-pas, très efficace.

5.2 Algorithmes de Contrôle (Q-Learning et SARSA)

Le but est d'apprendre les $Q(s, a)$ pour déduire la meilleure action $\max_a Q(s, a)$.

5.2.1 SARSA (On-Policy)

SARSA pour (State-Action-Reward-State-Action). L'agent apprend la valeur de la politique qu'il est en train de suivre *réellement*, y compris ses erreurs d'exploration.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

L'action A' est celle que l'agent va vraiment faire au tour suivant.

5.2.2 Q-Learning (Off-Policy)

C'est l'algorithme le plus célèbre. L'agent apprend la valeur de la politique **optimale**, peu importe ce qu'il fait dans la réalité. Pour la mise à jour, il ne regarde pas l'action A' qu'il va faire, mais la **meilleure action possible** $\max_{a'} Q(S', a')$.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$$

C'est très puissant car même si l'agent explore au hasard, il apprend la stratégie parfaite en arrière-plan.

Chapitre 6

Le Dilemme Exploration-Exploitation

C'est défi fondamental du RL. En arrivant dans un restaurant :

— **Exploitation** : Vous commandez votre plat préféré. Vous êtes sûr d'avoir un bon repas (maximisation immédiate), mais vous ne découvrirez jamais ce qu'il y a d'autre.

— **Exploration** : Vous commandez un plat au hasard. Vous risquez d'être déçu (reward négatif), mais vous pourriez découvrir un plat encore meilleur (découverte de l'optimum global).

Si on ne fait que l'Exploitation, on reste bloqué dans une solution sub-optimale. Si on ne fait que l'Exploration, on ne gagne jamais de points.

6.1 La Solution : Epsilon-Greedy

La méthode standard est d'utiliser un paramètre ϵ (epsilon).

— Avec une probabilité ϵ, l'agent choisit une action totalement **aléatoire**.

— Avec une probabilité $1 - \epsilon$, l'agent choisit l'action qu'il pense être la meilleure (**greedy**).

Généralement, on utilise un **Epsilon Decay** : on commence avec $\epsilon = 1$ (exploration totale) au début de l'apprentissage, et on le diminue progressivement vers 0.01 pour laisser place à l'exploitation une fois que l'agent est entraîné.

Chapitre 7

Mise en Pratique : Code Commenté

Voici une implémentation complète de Q-Learning sur l'environnement "FrozenLake" de la librairie Gymnasium.

```python
import gymnasium as gym
import numpy as np

# 1. Creation de l'environnement
# FrozenLake : Une grille glissante. But : aller de S a G sans tomber dans les trous H.
# is_slippery=True rend l'environnement stochastique (transitions incertaines)
env = gym.make("FrozenLake-v1", is_slippery=True)

# 2. Initialisation de la Q-Table
# Une matrice de taille [Nb_Etats, Nb_Actions] initialisee a 0
q_table = np.zeros((env.observation_space.n, env.action_space.n))

# 3. Hyperparametres
alpha = 0.1         # Learning Rate (Vitesse d'apprentissage)
gamma = 0.99        # Discount Factor (Importance du futur)
epsilon = 1.0       # Exploration Rate (100% aleatoire au debut)
epsilon_decay = 0.995 # On reduit l'exploration de 0.5% a chaque episode
min_epsilon = 0.01

episodes = 2000

for episode in range(episodes):
    state, info = env.reset() # On remet l'agent au depart
    done = False

    while not done:
        # --- Strategie Epsilon-Greedy ---
        if np.random.rand() < epsilon:
            action = env.action_space.sample() # Exploration (Hasard)
        else:
            action = np.argmax(q_table[state, :]) # Exploitation (Meilleur connu)

        # --- Interaction avec l'environnement ---
        next_state, reward, terminated, truncated, info = env.step(action)

        # --- Mise a jour Q-Learning (Formule Mathematique) ---
```

```
        # On regarde quelle est la valeur MAX possible au prochain etat
        max_next_q = np.max(q_table[next_state, :])

        # La valeur actuelle
        current_q = q_table[state, action]

        # Le calcul de l'erreur et la mise a jour
        # Q(s,a) = Q(s,a) + alpha * (Reward + gamma * max(Q(s')) - Q(s,a))
        new_q = current_q + alpha * (reward + gamma * max_next_q - current_q)

        q_table[state, action] = new_q

        # On avance
        state = next_state
        done = terminated or truncated

    # A la fin de l'episode, on reduit epsilon
    epsilon = max(min_epsilon, epsilon * epsilon_decay)

print("Apprentissage termine !")
print(q_table)
```